CS 216: Bitcoin Transaction Lab

Assignment Report — HashBandits

Team HashBandits

Kothalkar Bhagyesh Ritesh (240041024) Yash Bhamare (240041040)

Khush Kumar Singh (240041023) KVL Sarath Chandra (240001039)

March 10, 2026

Contents

1 Introduction **3**

2 Environment Setup **3**
2.1 Bitcoin Configuration . . . 3
2.2 Docker Setup . . . 3
2.3 RPC Connection . . . 4

3 Part 1: Legacy (P2PKH) Transactions **4**
3.1 Workflow Overview . . . 4
3.2 Transaction A → B . . . 4
3.2.1 TXIDs . . . 4
3.2.2 Decoded ScriptPubKey (Locking Script for B) . . . 4
3.3 Transaction B → C . . . 5
3.3.1 TXID . . . 5
3.3.2 Decoded ScriptSig (Unlocking Script) . . . 5
3.3.3 Challenge–Response Validation . . . 5
3.4 btcdeb Validation — Legacy . . . 6

4 Part 2: P2SH-SegWit (P2SH-P2WPKH) Transactions **7**
4.1 Workflow Overview . . . 7
4.2 Transaction A′ → B′ . . . 7
4.2.1 TXIDs . . . 7
4.2.2 Decoded ScriptPubKey (Locking Script for B′) . . . 7
4.3 Transaction B′ → C′ . . . 8
4.3.1 TXID . . . 8
4.3.2 Decoded ScriptSig . . . 8
4.3.3 Witness Stack . . . 8
4.3.4 Challenge–Response Validation . . . 8
4.4 btcdeb Validation — SegWit . . . 9

5 Part 3: Comparative Analysis **10**
5.1 Transaction Size Comparison 10
5.2 Script Structure Comparison 10
5.3 Why SegWit Transactions Are Smaller 10
5.3.1 1. The Witness Discount 10
5.3.2 2. Segregation of Signature Data 11
5.3.3 3. Practical Benefits 11

6 Conclusion **11**

1 Introduction

This report documents our implementation and analysis of Bitcoin transactions on a regtest network. We cover:

1. **Part 1:** Legacy (P2PKH) transactions — address generation, raw transaction creation, signing, broadcasting, and script validation using `btcdeb`.
2. **Part 2:** P2SH-SegWit (P2SH-P2WPKH) transactions — same workflow with segregated witness addresses.
3. **Part 3:** Comparative analysis — transaction size, script structure, and the benefits of SegWit.

All code was run inside Docker containers orchestrated with Docker Compose. Bitcoin Core v30.2.0 was used in regtest mode, and the client was built with `python-bitcoinlib` and `btcdeb`.

2 Environment Setup

2.1 Bitcoin Configuration

The `bitcoin.conf` file configures the regtest node with the following parameters:

```
regtest=1
server=1
printtoconsole=1

[regtest]
rpcuser=test
rpcpassword=test123
paytxfee=0.0001
fallbackfee=0.00001
mintxfee=0.00001
txconfirmtarget=6
txindex=1
```

2.2 Docker Setup

The project uses a multi-stage Dockerfile that:

- Compiles `btcdeb` from source (for script debugging)
- Installs `python-bitcoinlib` and `pexpect` via pip
- Copies the source code to `/app/src/`

Docker Compose orchestrates two services: `bitcoin_server` (Bitcoin Core) and `client` (our Python program).

2.3 RPC Connection

Our program connects to `bitcoind` via JSON-RPC using the credentials from environment variables:

```
base_url = f"http://{user}:{password}@{host}:{port}"
node_conn = Proxy(service_url=base_url)
```

3 Part 1: Legacy (P2PKH) Transactions

3.1 Workflow Overview

1. Connect to `bitcoind` via RPC and create/load a wallet
2. Generate three legacy (P2PKH) addresses: A, B, and C
3. Fund address A using `sendtoaddress`
4. Create a raw transaction A → B, decode it to extract the ScriptPubKey
5. Sign with `signrawtransactionwithwallet` and broadcast
6. Use `listunspent` to find B's UTXO from the A → B transaction
7. Create, sign, and broadcast B → C
8. Validate scripts using `btcdeb`

3.2 Transaction A → B

3.2.1 TXIDs

- **Funding TXID:** `a79ad17d8cb47759360db4b335123f8a89ca0588be74861cf69854ddfb648331`
- **A → B TXID:** `f1bc8b9edda359cbcbf4e84f65648e4d341446c7d68e893e5837f58554e6b22c`

3.2.2 Decoded ScriptPubKey (Locking Script for B)

The ScriptPubKey locked by the A → B transaction for address B:

```
[A → B (signed)]
  ScriptPubKey (locking / challenge):
    type : pubkeyhash
    asm  : OP_DUP OP_HASH160 b2db8196abcc692c109310a2a1e6c65473485bec OP_EQUALVERIFY OP_CHECKSIG
    hex  : 76a914b2db8196abcc692c109310a2a1e6c65473485bec88ac
  ScriptSig (unlocking / response):
    asm  : 304402201a1f022a6474c869011fad24f7481b4a6e5b377a44fe2c9846c4871c89aaa87b02205924ad1487f02bd8cac2554f94
ed10847c02eadec4635b4fa69bddd378858e5c[ALL] 0243de7315f391e87b09a07801d14eff333ff6e24bba13bf58e3b0f31c3319d299
    hex  : 47304402201a1f022a6474c869011fad24f7481b4a6e5b377a44fe2c9846c4871c89aaa87b02205924ad1487f02bd8cac2554f
94ed10847c02eadec4635b4fa69bddd378858e5c01210243de7315f391e87b09a07801d14eff333ff6e24bba13bf58e3b0f31c3319d299
```

This is a standard P2PKH script. It challenges the spender to provide a public key that hashes to `7d9018...b228` and a valid signature.

3.3 Transaction B → C

3.3.1 TXID

- **B → C TXID:** `58f497b1085821dd539d6fc6fb28b8219102a9e7b7c01f024906340d90227b1a`
- **Input UTXO:** The output from A → B (vout 0)

3.3.2 Decoded ScriptSig (Unlocking Script)

```
[B → C (signed)]
  ScriptPubKey (locking / challenge):
    type : pubkeyhash
    asm  : OP_DUP OP_HASH160 cb2f14ad5036edd09da61fdeadeb35bb53959ebe OP_EQUALVERIFY OP_CHECKSIG
    hex  : 76a914cb2f14ad5036edd09da61fdeadeb35bb53959ebe88ac
  ScriptSig (unlocking / response):
    asm  : 304402206120627b033fd6bbcde87c65ff3c4dae9aed1663d5b3f23292150eaff1a5e7e9022012db48f0591553e7528d97b90e
15e01b38facc9a8980494cdd101034a6b23553[ALL] 03feb3f6b37b8a357ef6179dab388bb6f1b2ae0777b0cf54672603e5f35035b1ec
    hex  : 47304402206120627b033fd6bbcde87c65ff3c4dae9aed1663d5b3f23292150eaff1a5e7e9022012db48f0591553e7528d97b9
0e15e01b38facc9a8980494cdd101034a6b23553012103feb3f6b37b8a357ef6179dab388bb6f1b2ae0777b0cf54672603e5f35035b1ec
```

The ScriptSig contains two data pushes:

1. The **ECDSA signature** (71 bytes) with SIGHASH_ALL flag
2. The **public key** (33 bytes, compressed)

3.3.3 Challenge–Response Validation

When Bitcoin validates this transaction, it concatenates the ScriptSig with the ScriptPubKey and executes:

```
<Sig> <PubKey> OP_DUP OP_HASH160 <PubKeyHash> OP_EQUALVERIFY
   OP_CHECKSIG
```

1. Push `<Sig>` and `<PubKey>` onto the stack
2. `OP_DUP`: Duplicate the public key
3. `OP_HASH160`: Hash the duplicate → produces pubkey hash
4. Push the expected hash from ScriptPubKey
5. `OP_EQUALVERIFY`: Compare the two hashes — must match
6. `OP_CHECKSIG`: Verify the signature against the public key
7. Result: `01` (TRUE) — transaction is valid

3.4 btcdeb Validation — Legacy

We ran `btcdeb` with the signed B → C transaction and the previous A → B transaction as input. The debugger stepped through all 8 opcodes:

```
btcdeb 5.0.24 -- type `/usr/local/bin/btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
got transaction 8152fab8584ed8d2743461950fddcc2003b3ec0eed23712de0cc2baddc199058:
CTransaction(hash=8152fab858, ver=2, vin.size=1, vout.size=1, nLockTime=0)
    CTxIn(COutPoint(fceea5b813, 0), scriptSig=4730440220266eed3d6889bb, nSequence=4294967293)
    CScriptWitness()
    CTxOut(nValue=4.99980000, scriptPubKey=76a914a92fae76ae98c581d54dc6ad)

got input tx #0 fceea5b813ba435f87b573f8e08c6d6e77103c7297b020571e62fdf311fcf883:
CTransaction(hash=fceea5b813, ver=2, vin.size=1, vout.size=1, nLockTime=0)
    CTxIn(COutPoint(7c0f01bae8, 1), scriptSig=473044022077645d4e6cd591, nSequence=4294967293)
    CScriptWitness()
    CTxOut(nValue=4.99990000, scriptPubKey=76a914871853f61dad1cf54bfcb460)

input tx index = 0; tx input vout = 0; value = 499990000
got witness stack of size 0
8 op script loaded. type `help` for usage information
script                                                          |  stack
----------------------------------------------------------------+--------
30440220266eed3d6889bb4dada1b79769c2b75f98ae14f59e41eb393f71b8e... |
03a880ea4566794a8eddad261eff9eecc700d22f411c336e0dcb4d78ee6f2622e5 |
<<< scriptPubKey >>>                                            |
OP_DUP                                                          |
OP_HASH160                                                      |
871853f61dad1cf54bfcb4608118816a985df125                        |
OP_EQUALVERIFY                                                  |
OP_CHECKSIG                                                     |
#0000 30440220266eed3d6889bb4dada1b79769c2b75f98ae14f59e41eb393f71b8e78d664a1b022073b36de430a05268ea841fd8123ab470067bdaae7a3f18c7011bfe27f8e04a2701

--- Initial btcdeb State loaded successfully ---

--- Executing Step 1 ---
step
                <> PUSH stack 30440220266eed3d6889bb4dada1b79769c2b75f98ae14f59e41eb393f71b8e78d664a1b022073b36de430a05268ea841fd8123ab470067bdaae7a3f18c7011bfe27f8e04a2701
btcdeb> script                                                  |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
03a880ea4566794a8eddad261eff9eecc700d22f411c336e0dcb4d78ee6f2622e5 | 30440220266eed3d6889bb4dada1b79769c2b75f98ae14f59e41eb393f71b8e...
<<< scriptPubKey >>>                                            |
OP_DUP                                                          |
OP_HASH160                                                      |
871853f61dad1cf54bfcb4608118816a985df125                        |
OP_EQUALVERIFY                                                  |
OP_CHECKSIG                                                     |
#0001 03a880ea4566794a8eddad261eff9eecc700d22f411c336e0dcb4d78ee6f2622e5

--- Executing Step 2 ---
step
                <> PUSH stack 03a880ea4566794a8eddad261eff9eecc700d22f411c336e0dcb4d78ee6f2622e5
btcdeb> script                                                  |                                                                stack
```

```
--- Executing Step 2 ---
step
                <> PUSH stack 03a880ea4566794a8eddad261eff9eecc700d22f411c336e0dcb4d78ee6f2622e5
btcdeb> script                                                  |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
<<< scriptPubKey >>>                                            | 03a880ea4566794a8eddad261eff9eecc700d22f411c336e0dcb4d78ee6f2622e5
OP_DUP                                                          | 30440220266eed3d6889bb4dada1b79769c2b75f98ae14f59e41eb393f71b8e...
OP_HASH160                                                      |
871853f61dad1cf54bfcb4608118816a985df125                        |
OP_EQUALVERIFY                                                  |
OP_CHECKSIG                                                     |
<<< scriptPubKey >>>

--- Executing Step 3 ---
step
btcdeb> script                                                  |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
OP_DUP                                                          | 03a880ea4566794a8eddad261eff9eecc700d22f411c336e0dcb4d78ee6f2622e5
OP_HASH160                                                      | 30440220266eed3d6889bb4dada1b79769c2b75f98ae14f59e41eb393f71b8e...
871853f61dad1cf54bfcb4608118816a985df125                        |
OP_EQUALVERIFY                                                  |
OP_CHECKSIG                                                     |
#0003 OP_DUP

--- Executing Step 4 ---
step
                <> PUSH stack 03a880ea4566794a8eddad261eff9eecc700d22f411c336e0dcb4d78ee6f2622e5
btcdeb> script                                                  |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
OP_HASH160                                                      | 03a880ea4566794a8eddad261eff9eecc700d22f411c336e0dcb4d78ee6f2622e5
871853f61dad1cf54bfcb4608118816a985df125                        | 03a880ea4566794a8eddad261eff9eecc700d22f411c336e0dcb4d78ee6f2622e5
OP_EQUALVERIFY                                                  | 30440220266eed3d6889bb4dada1b79769c2b75f98ae14f59e41eb393f71b8e...
OP_CHECKSIG                                                     |
#0004 OP_HASH160

--- Executing Step 5 ---
step
                <> POP  stack
                <> PUSH stack 871853f61dad1cf54bfcb4608118816a985df125
btcdeb> script                                                  |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
871853f61dad1cf54bfcb4608118816a985df125                        |                                871853f61dad1cf54bfcb4608118816a985df125
OP_EQUALVERIFY                                                  | 03a880ea4566794a8eddad261eff9eecc700d22f411c336e0dcb4d78ee6f2622e5
OP_CHECKSIG                                                     | 30440220266eed3d6889bb4dada1b79769c2b75f98ae14f59e41eb393f71b8e...
#0005 871853f61dad1cf54bfcb4608118816a985df125

--- Executing Step 6 ---
step
                <> PUSH stack 871853f61dad1cf54bfcb4608118816a985df125
btcdeb> script                                                  |                                                                stack
```

```
--- Executing Step 6 ---
step
                <> PUSH stack 871853f61dad1cf54bfcb4608118816a985df125

btcdeb> script                                                          |                                                
```

4.3 Transaction B′ → C′

4.3.1 TXID

- **B′ → C′ TXID:** `5860acbefb867ed29a0066e66099deebe8e735e1e6a30d14f6359f033af3edc7`

4.3.2 Decoded ScriptSig

```
[B' → C' (signed)]
  ScriptPubKey (locking / challenge):
    type : scripthash
    asm  : OP_HASH160 0aa4d29b86a283c783c60542fda02832219cdb23 OP_EQUAL
    hex  : a9140aa4d29b86a283c783c60542fda02832219cdb2387
  ScriptSig (unlocking / response):
    asm  : 001464eb750d9c54355cb5cb87443e48466c3c90ede9
    hex  : 16001464eb750d9c54355cb5cb87443e48466c3c90ede9
  Witness Stack (2 items):
    [0]: 304402205ee995ae989f934e92beb36542ad656491f54f1d83b4ecd655ffcc5ecab3bfc8022056e36b447dd0c868b0b526bfd4c4
7b575d0a83d568a0471ddb516b596231889001
    [1]: 0390594e86f12e7fc68118af2bb9a9fefe12f3c5f2e5721a98bdece634c736e227
```

Notice how short this is compared to Legacy! The ScriptSig **only contains the redeem script** (a P2WPKH witness program: version byte `00` + 20-byte pubkey hash). The actual signature and public key are in the witness.

4.3.3 Witness Stack

```
Witness[0]: 304402202a7be8cfa90b083ded42e344f1f8dd628409c905
            e996b2e6a0a0a111fe5c9af502205af06bfbda5b6988c7f1
            95d1a93213c243a592ef888e36dad048a1e2a5592ae401
Witness[1]: 02273959e57772b1554f5f99421d117b8ecb4005b0b385
            432ee898e854192f83d8
```

- **Witness[0]:** The ECDSA signature (71 bytes)
- **Witness[1]:** The compressed public key (33 bytes)

4.3.4 Challenge–Response Validation

For P2SH-P2WPKH, validation happens in two stages:

1. **P2SH stage:** The ScriptSig's redeem script is hashed and compared against the ScriptPubKey hash — they must match.
2. **Witness stage:** The redeem script is recognized as a witness program. The witness data (sig + pubkey) is used in a virtual P2PKH script evaluation, identical to Legacy but with the SegWit sighash algorithm.

4.4 btcdeb Validation — SegWit

```
btcdeb 5.0.24 -- type `/usr/local/bin/btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
got segwit transaction eafbeb90e72f0a2648b7195525ee1675d5b20eebe192dace4892ec94379464f1:
CTransaction(hash=eafbeb90e7, ver=2, vin.size=1, vout.size=1, nLockTime=0)
    CTxIn(COutPoint(acc7968f98, 0), scriptSig=160014990951c69919750f20, nSequence=4294967293)
    CScriptWitness(304402201832cb8fa6548851a18ff6529ca30d751abbb6c0635e17e0f052b743a4fd53f3022059f89825c09b1b0fd1f35abe651d3f46f383b08955c6e9279bfa89bfa662977e01, 037ddfa41a41b63e434a4072bc5590ecb1cb5d75996c177d9ef1d2f77fbf39428a)
    CTxOut(nValue=4.99980000, scriptPubKey=a914f1d8559d3fbb690fd5e4a2afe0)

got input tx #0 acc7968f98f1ca416452bb4f1d425479fbf357a7d758bc755af62a1bbf2622d9:
CTransaction(hash=acc7968f98, ver=2, vin.size=1, vout.size=1, nLockTime=0)
    CTxIn(COutPoint(c34af10e4f, 1), scriptSig=160014c4e35b7d00fd428303, nSequence=4294967293)
    CScriptWitness(304402201929d4ded99b34af38b0a6b2f22b8e2d954ea29e3295aa06c5368f895619c91602202bf457f7e59d4f8759cd41698e84ff6735d8f908cd33b015c599566b2c7a6a1c01, 038e4a404ec11368fae2e437510a46668055839f2f0402ce34b56c90c52c9b0919)
    CTxOut(nValue=4.99990000, scriptPubKey=a914677be775452f285b4e9310d885)

input tx index = 0; tx input vout = 0; value = 499990000
got witness stack of size 2
script sig non-empty; embedded P2SH (extracting payload)
hash source = 0014990951c69919750f20a32725cff0676933ae26c5
22 bytes (P2WPKH)
valid script
- generating prevout hash from 1 ins
[+] COutPoint(acc7968f98, 0)
*** note: there is a for-clarity preamble

This is a virtual script that btcdeb generates and presents to you so you can step through the validation process one step at a time. The input is simply the redeem script hash, whereas btcdeb presents it as a OP_DUP, OP_HASH160, <that
 hash>, OP_EQUALVERIFY script.
5 op script loaded. type `help` for usage information
script                                   |                                                          stack
-----------------------------------------+----------------------------------------------------------
OP_DUP                                   | 037ddfa41a41b63e434a4072bc5590ecb1cb5d75996c177d9ef1d2f77fbf39428a
OP_HASH160                               | 304402201832cb8fa6548851a18ff6529ca30d751abbb6c0635e17e0f052b74...
990951c69919750f20a32725cff0676933ae26c5 |
OP_EQUALVERIFY                           |
OP_CHECKSIG                              |
#0000 OP_DUP

--- Initial btcdeb State loaded successfully ---

--- Executing Step 1 ---
step
                <> PUSH stack 037ddfa41a41b63e434a4072bc5590ecb1cb5d75996c177d9ef1d2f77fbf39428a
btcdeb> script                           |                                                          stack
-----------------------------------------+----------------------------------------------------------
OP_HASH160                               | 037ddfa41a41b63e434a4072bc5590ecb1cb5d75996c177d9ef1d2f77fbf39428a
990951c69919750f20a32725cff0676933ae26c5 | 037ddfa41a41b63e434a4072bc5590ecb1cb5d75996c177d9ef1d2f77fbf39428a
OP_EQUALVERIFY                           | 304402201832cb8fa6548851a18ff6529ca30d751abbb6c0635e17e0f052b74...
OP_CHECKSIG                              |
#0001 OP_HASH160

--- Executing Step 2 ---
step
                <> POP  stack
```

```
--- Executing Step 2 ---
step
                <> POP  stack
                <> PUSH stack 990951c69919750f20a32725cff0676933ae26c5
btcdeb> script                           |                                                          stack
-----------------------------------------+----------------------------------------------------------
990951c69919750f20a32725cff0676933ae26c5 |                                990951c69919750f20a32725cff0676933ae26c5
OP_EQUALVERIFY                           | 037ddfa41a41b63e434a4072bc5590ecb1cb5d75996c177d9ef1d2f77fbf39428a
OP_CHECKSIG                              | 304402201832cb8fa6548851a18ff6529ca30d751abbb6c0635e17e0f052b74...
#0002 990951c69919750f20a32725cff0676933ae26c5

--- Executing Step 3 ---
step
                <> PUSH stack 990951c69919750f20a32725cff0676933ae26c5
btcdeb> script                           |                                                          stack
-----------------------------------------+----------------------------------------------------------
OP_EQUALVERIFY                           |                                990951c69919750f20a32725cff0676933ae26c5
OP_CHECKSIG                              |                                990951c69919750f20a32725cff0676933ae26c5
                                         | 037ddfa41a41b63e434a4072bc5590ecb1cb5d75996c177d9ef1d2f77fbf39428a
                                         | 304402201832cb8fa6548851a18ff6529ca30d751abbb6c0635e17e0f052b74...
#0003 OP_EQUALVERIFY

--- Executing Step 4 ---
step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
btcdeb> script                           |                                                          stack
-----------------------------------------+----------------------------------------------------------
OP_CHECKSIG                              | 037ddfa41a41b63e434a4072bc5590ecb1cb5d75996c177d9ef1d2f77fbf39428a
                                         | 304402201832cb8fa6548851a18ff6529ca30d751abbb6c0635e17e0f052b74...
#0004 OP_CHECKSIG

--- Executing Step 5 ---
step
EvalChecksig() sigversion=1
Eval Checksig Pre-Tapscript
GenericTransactionSignatureChecker::CheckECDSASignature(71 len sig, 33 len pubkey, sigversion=1)
  sig         = 304402201832cb8fa6548851a18ff6529ca30d751abbb6c0635e17e0f052b743a4fd53f3022059f89825c09b1b0fd1f35abe651d3f46f383b08955c6e9279bfa89bfa662977e01
  pub key     = 037ddfa41a41b63e434a4072bc5590ecb1cb5d75996c177d9ef1d2f77fbf39428a
  script code = 76a914990951c69919750f20a32725cff0676933ae26c588ac
  hash type   = 01 (SIGHASH_ALL)
SignatureHash(nIn=0, nHashType=01, amount=499990000)
- sigversion == SIGVERSION_WITNESS_V0
  sighash     = 11f59eea0bc0325ab5fd753c4577afec5266398ce1f6b8884796c102bb4c95e0
  pubkey.VerifyECDSASignature(sig=304402201832cb8fa6548851a18ff6529ca30d751abbb6c0635e17e0f052b743a4fd53f3022059f89825c09b1b0fd1f35abe651d3f46f383b08955c6e9279bfa89bfa662977e, sighash=11f59eea0bc0325ab5fd753c4577afec5266398ce1f6b888479
6c102bb4c95e0):
  result: success
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
btcdeb> script                           |                                                          stack
```

Note the `sigversion = SIGVERSION_WITNESS_V0`, confirming SegWit v0 script evaluation with the BIP-143 sighash algorithm.

5 Part 3: Comparative Analysis

5.1 Transaction Size Comparison

Transaction	Size (bytes)	vSize (vbytes)	Weight (WU)
Legacy A → B	191	191	764
SegWit A′ → B′	215	134	533
Legacy B → C	191	191	764
SegWit B′ → C′	215	134	533

Table 1: Transaction size comparison: Legacy vs P2SH-SegWit

Metric	Legacy	SegWit	Difference
Size (bytes)	191	215	SegWit is 24 bytes *larger*
vSize (vbytes)	191	134	SegWit saves 57 vbytes
Weight (WU)	764	533	SegWit saves 231 WU
vSize Saving		**29.8%**	

Table 2: Delta comparison (B → C vs B′ → C′)

5.2 Script Structure Comparison

Component	Legacy (P2PKH)	SegWit (P2SH-P2WPKH)
ScriptPubKey	`OP_DUP OP_HASH160` `<PubKeyHash>` `OP_EQUALVERIFY` `OP_CHECKSIG` (25 bytes)	`OP_HASH160` `<RedeemScriptHash>` `OP_EQUAL` (23 bytes)
ScriptSig	`<Sig> <PubKey>` (∼107 bytes)	`<RedeemScript>` (∼23 bytes)
Witness	N/A	`<Sig> <PubKey>` (∼107 bytes, counted at 1/4 weight)

Table 3: Script structure comparison

5.3 Why SegWit Transactions Are Smaller

Although the raw byte size of SegWit transactions is actually *larger* (215 vs 191 bytes), the **virtual size** is significantly smaller (134 vs 191 vbytes). This is due to three factors:

5.3.1 1. The Witness Discount

Bitcoin's block weight limit (4,000,000 WU) counts witness data at one-quarter the rate of non-witness data:

$$\text{weight} = (\text{non-witness bytes} \times 4) + (\text{witness bytes} \times 1) \tag{1}$$

$$\text{vsize} = \lceil \text{weight}/4 \rceil \tag{2}$$

Since signatures and public keys (∼104 bytes) are moved to the witness section, they are counted at only 25% of their actual size when computing vsize.

5.3.2 2. Segregation of Signature Data

In Legacy P2PKH, the ScriptSig embeds both the signature (∼71 bytes) and public key (∼33 bytes) directly in the transaction body, all counted at full weight. In P2SH-P2WPKH, the ScriptSig is reduced to just the 23-byte redeem script, while the signature and public key are separated into the witness field.

5.3.3 3. Practical Benefits

- **Lower fees:** Miners charge based on vsize, so the 29.8% reduction directly translates to lower transaction fees.
- **Higher throughput:** The 4 MW block weight limit accommodates more SegWit transactions than legacy ones, increasing effective block capacity.
- **Malleability fix:** Moving signatures out of the transaction hash input prevents third-party transaction malleability, enabling second-layer protocols like the Lightning Network.

6 Conclusion

We successfully implemented both Legacy (P2PKH) and P2SH-SegWit (P2SH-P2WPKH) transaction chains on a Bitcoin regtest network. The key takeaways are:

- Legacy transactions use straightforward `OP_DUP OP_HASH160 ... OP_CHECKSIG` scripts with signatures inline in ScriptSig.
- P2SH-SegWit transactions wrap a witness program in a P2SH address, moving signatures to the witness for a 29.8% virtual size reduction.
- The witness discount incentivizes SegWit adoption through lower fees and higher block throughput.
- Both transaction types were validated end-to-end using `btcdeb`, confirming successful script execution.

The complete terminal outputs are provided as text files (`legacy.txt`, `segwit.txt`, `comparison.txt`) in the repository root, captured using `| tee` for full traceability.